Full Audit: Project Health Pulse Run

Stage timing, token consumption and cost for one end-to-end pipeline run

Dr. Anish Roychowdhury | Generative AI Architect & Educator

6 October 2026

Total run time	API calls	Tokens (in / out)	Cost
2m 55s	5	29,371 / 10,057	$0.70

1 Stage Timing

Total wall-clock time for the run was 2 minutes 55 seconds. The three language-model stages account for almost all of it; the four code stages each completed in under a millisecond.

Stage	What it does	Time
S0 Ingest	Parse and normalise 5 transcripts	<1 ms
S1 Extractor	LLM reads all transcripts and rates 9 items	59.8 s
S2 Grounding	Fuzzy-match all quotes against source lines	<1 ms
S3 Debate	Challenger, Defender and Judge across 9 items	60.6 s
S4 Roll-up	Worst-of pillar computation	<1 ms
S5 Recommender	Draft commentary and recommendations (2 API calls†)	54.4 s
S6 Assemble	Merge all stages into `final.json`	<1 ms
Total		**2m 55s**

Table 1: Wall-clock time per pipeline stage.

2 Token Consumption

Call	Model	Input	Output
S1 Extractor	sonnet-4-5	24	3,117
S3a Challenger	sonnet-4-5	5,043	669
S3b Defender	sonnet-4-5	3,995	458
S3c Judge	opus-4-5	6,366	2,658
S5 Recommender†	sonnet-4-5	13,943	3,155
Totals		**29,371**	**10,057**
Cache written	sonnet	57,999	n/a
Cache read	sonnet	11,401	n/a

Table 2: Tokens per API call. Cache tokens are reported separately from regular input.

3 Cost and Prompt Caching

Total cost for the run was **$0.70**.

The Extractor shows only 24 tokens of regular input because the full 57,999-token transcript block was written to the prompt cache on that first call. Later Sonnet stages (Challenger, Defender, Recommender) read from the cache at $0.30 per million tokens rather than the standard $3.00. However, writing to the cache costs $3.75 per million tokens, a 25% premium over standard input. On this single run the net saving from caching is therefore slightly negative, at −$0.013.

Key Idea

Caching pays off from roughly the second run onwards, once the cache is already warm and the write premium is not incurred again.

4 Validation Retry in the Recommender

† On its first attempt the Recommender exceeded the 120-word limit on pillar commentary, returning 139, 141 and 132 words for the three pillars. The validation error was fed back to the model, and it corrected the output on the retry. This is the repair-retry mechanism in `llm.py` working as designed. Tokens from both calls are included in the Recommender's 13,943 input and 3,155 output figures.

References

[1] Anthropic. *Prompt caching*. Anthropic Documentation. `https://docs.claude.com/en/docs/build-with-claude/prompt-caching`

[2] Anthropic. *Pricing*. Anthropic Documentation. `https://docs.claude.com/en/docs/about-claude/pricing`

[3] Anthropic. *Tool use with Claude*. Anthropic Documentation. `https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview`